

Extended Kalman Filter for GPS/INS Fusion

Module Documentation: `kalman.py`

GPS Track Conditioning Pipeline

Contents

1	Introduction	2
1.1	Design Goals	2
2	State-Space Formulation	2
2.1	State Vector	2
2.2	Motion Model: CTRV	2
2.3	Jacobian Matrix $\mathbf{F}$	3
3	Process Noise	3
3.1	Standard Process Noise Matrix $\mathbf{Q}$	3
3.2	Position Drift Fix (Critical for High Sample Rates)	3
4	Position Pre-Filtering	4
5	Measurement Updates	4
5.1	GPS Position (Sparse, 5–20 Hz)	4
5.2	Wheel Speed (Dense, Full Rate)	5
5.3	Yaw Rate (Dense, Full Rate)	5
6	Rauch–Tung–Striebel Smoother	5
7	Coordinate System	6
7.1	WGS84 $\leftrightarrow$ Local ENU Conversion	6
8	Implementation Architecture	6
8.1	Numba JIT Compilation	6
8.2	Class Structure	7
8.3	JAX Backend Parity	7
9	Tuning Parameters	8
9.1	Speed Bias from RTS Smoothing	8
10	Data Flow	8
10.1	Input Requirements	8
10.2	Output: <code>KalmanResult</code>	9
11	ABS Braking Robustness	9
11.1	Stage 1: Speed Interpolation	9
11.2	Stage 2: Wheel-Speed Measurement-Noise Inflation	9
12	GPS Freeze Repair (Preprocessing)	11
12.1	Phenomenon	11
12.2	Detection: Innovation-Based Criterion	11
12.3	Repair Strategy: Linear Interpolation	11
12.4	Why Not Partial (1D) Measurement Updates?	12
12.5	Code Reference	12
13	Summary	13
	References	14

1 Introduction

The `kalman.py` module implements a sensor-fusion algorithm that combines three independent measurement sources—GPS position, wheel speed, and yaw rate—into a single, optimal state estimate of a vehicle’s trajectory. The core algorithm is an **Extended Kalman Filter** (EKF) with a nonlinear motion model, augmented by a **Rauch–Tung–Striebel** (RTS) smoother [1] for offline batch processing.

The module is designed for high-rate vehicle measurement data (sampling rate $f_s \approx 500$ Hz) where GPS updates occur sparsely at 5–20 Hz. Between GPS fixes, the filter propagates position using wheel speed and yaw rate (dead reckoning), then corrects when a GPS measurement arrives.

1.1 Design Goals

- **Accuracy:** Sub-meter position accuracy by fusing complementary sensors
- **Robustness:** Graceful handling of GPS dropouts via dead reckoning
- **Performance:** Real-time capable via Numba JIT compilation (14× speedup over Python)
- **Modularity:** Pure-function math core enabling future JAX port

2 State-Space Formulation

2.1 State Vector

The filter maintains a five-dimensional state vector:

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ v \\ \theta \\ \omega \end{pmatrix} \quad (1)$$

where:

- x, y — position in a local East-North-Up (ENU) frame [m]
- v — scalar longitudinal speed [m/s]
- θ — heading angle ($0 = \text{East}$, $\pi/2 = \text{North}$) [rad]
- ω — yaw rate (angular velocity about vertical axis) [rad/s]

The local ENU frame is centered at the first GPS fix, with the x -axis pointing East and the y -axis pointing North. This avoids numerical issues from operating in WGS84 coordinates directly.

2.2 Motion Model: CTRV

The **Constant Turn Rate and Velocity** (CTRV) model assumes that between time steps, the vehicle maintains constant speed v and constant yaw rate ω . The resulting trajectory is a circular arc.

For $\omega \neq 0$:

$$x_{k+1} = x_k + \frac{v_k}{\omega_k} (\sin(\theta_k + \omega_k \Delta t) - \sin \theta_k) \quad (2)$$

$$y_{k+1} = y_k + \frac{v_k}{\omega_k} (\cos \theta_k - \cos(\theta_k + \omega_k \Delta t)) \quad (3)$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t \quad (4)$$

For $\omega \approx 0$ (straight-line motion):

$$x_{k+1} = x_k + v_k \cos(\theta_k) \cdot \Delta t \quad (5)$$

$$y_{k+1} = y_k + v_k \sin(\theta_k) \cdot \Delta t \quad (6)$$

Speed and yaw rate persist: $v_{k+1} = v_k$, $\omega_{k+1} = \omega_k$.

This model is **nonlinear** in the state—the position depends on trigonometric functions of θ and ω —which is why a standard (linear) Kalman filter cannot be used. The EKF linearizes this model at each step.

2.3 Jacobian Matrix F

The EKF requires the Jacobian of the state transition function $\mathbf{f}(\mathbf{x})$ with respect to the state. For $\omega \neq 0$:

$$\mathbf{F} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} 1 & 0 & \frac{\sin(\theta+\omega\Delta t) - \sin \theta}{\omega} & \dots & \dots \\ 0 & 1 & \frac{\cos \theta - \cos(\theta+\omega\Delta t)}{\omega} & \dots & \dots \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad (7)$$

The full expressions for $F_{0,3}, F_{0,4}, F_{1,3}, F_{1,4}$ involve partial derivatives of the circular-arc equations with respect to θ and ω , computed analytically in the source code.

3 Process Noise

3.1 Standard Process Noise Matrix Q

The process noise models unmodeled accelerations (longitudinal σ_a) and yaw acceleration (σ_ω). The resulting covariance matrix has the block structure:

$$\mathbf{Q} = \begin{pmatrix} \Delta \frac{t^4}{4} \cdot \sigma_a^2 + \sigma_{\text{drift}}^2 \cdot \Delta t & 0 & \Delta \frac{t^3}{2} \cdot \sigma_a^2 & 0 & 0 \\ 0 & \Delta \frac{t^4}{4} \cdot \sigma_a^2 + \sigma_{\text{drift}}^2 \cdot \Delta t & \Delta \frac{t^3}{2} \cdot \sigma_a^2 & 0 & 0 \\ \Delta \frac{t^3}{2} \cdot \sigma_a^2 & \Delta \frac{t^3}{2} \cdot \sigma_a^2 & \Delta t^2 \cdot \sigma_a^2 & 0 & 0 \\ 0 & 0 & 0 & \Delta t^2 \cdot \sigma_\omega^2 & \Delta t \cdot \sigma_\omega^2 \\ 0 & 0 & 0 & \Delta t \cdot \sigma_\omega^2 & \sigma_\omega^2 \end{pmatrix} \quad (8)$$

3.2 Position Drift Fix (Critical for High Sample Rates)

At $\Delta t = 2$ ms (500 Hz), the standard position noise term $\Delta t^4/4 \cdot \sigma_a^2 \approx 4 \times 10^{-12}$ m² per step. Between GPS updates—typically 25 to 100 steps depending on whether GPS runs at 20 Hz or 5 Hz—the position covariance $\mathbf{P}_{0:1,0:1}$ barely grows. The Kalman gain for GPS corrections approaches zero, causing unbounded dead-reckoning drift.

The severity depends on the ratio f_s/f_{GPS} . At 500 Hz with 5 Hz GPS, 100 prediction steps occur between corrections. At 500 Hz with 20 Hz GPS, only 25 steps elapse. In both cases, the Δt^4 scaling is insufficient.

Fix: An additional term $\sigma_{\text{drift}}^2 \cdot \Delta t$ is added to $Q_{0,0}$ and $Q_{1,1}$. This scales **linearly** with Δt (not Δt^4), ensuring adequate position uncertainty growth:

$$\mathbf{P}_{\text{pos}} \text{ after } N \text{ steps} \approx N \times \sigma_{\text{drift}}^2 \times \Delta t \quad (9)$$

For example, with $N = 100$ steps (5 Hz GPS at 500 Hz sample rate):

$$P_{\text{pos}} \approx 100 \times 25 \times 0.002 = 5.0\text{m}^2 \quad (10)$$

$$K_{\text{GPS}} = \frac{P_{\text{pos}}}{P_{\text{pos}} + \sigma_{\text{GPS}}^2} = \frac{5.0}{5.0 + 4.0} \approx 0.56 \quad (11)$$

With $N = 25$ steps (20 Hz GPS):

$$P_{\text{pos}} \approx 25 \times 25 \times 0.002 = 1.25\text{m}^2 \quad (12)$$

$$K_{\text{GPS}} = \frac{1.25}{1.25 + 4.0} \approx 0.24 \quad (13)$$

Without this fix, $K_{\text{GPS}} \approx 0$ in both cases and GPS measurements are effectively ignored.

σ_{drift}	Median error	P95	Max
0 (broken)	> 100 m	—	—
5.0 (fixed)	1.4 m	3.4 m	6.6 m

Table 1: Effect of position drift noise on filter accuracy (500 Hz, 14 Hz GPS).

Rule of thumb: For any EKF where $f_s \gg f_{\text{GPS}}$, position process noise must scale with Δt (not Δt^4) to maintain GPS observability. The critical parameter is the number of prediction steps between GPS fixes: $N = f_s/f_{\text{GPS}}$.

4 Position Pre-Filtering

Before the interpolated GPS positions reach `GPSKalmanFilter.run()`, `GPSProcessor.process()` applies a zero-phase Butterworth low-pass (`filtfilt`, order `filter_order`, default 2) to `lon_interp/lat_interp`. The cutoff is set below the GPS update rate,

$$f_{\text{cutoff}} = 0.4 \cdot f_{\text{GPS}}, \quad f_{\text{GPS}} = N_{\text{updates}}/(n \cdot \Delta t), \quad (14)$$

i.e. a lower fraction than the `cutoff_factor` used for the pure Butterworth pipeline (default 0.5), to avoid overshoot feeding into the EKF. This pre-filtering step is not part of the mathematical filter description above — it changes the effective input noise seen by the GPS measurement update relative to a raw, merely S&H-interpolated signal, and should be kept in mind when tuning `sigma_pos_gps`.

5 Measurement Updates

The filter processes three measurement types at each time step, applied sequentially (scalar updates for efficiency):

5.1 GPS Position (Sparse, 5–20 Hz)

When a GPS fix is available (indicated by a boolean mask):

$$z_{\text{GPS}} = \begin{pmatrix} x_{\text{GPS}} \\ y_{\text{GPS}} \end{pmatrix}, \quad H_{\text{GPS}} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix} \quad (15)$$

The GPS measurement noise is **speed-dependent** to suppress multipath jitter at standstill:

$$\sigma_{\text{GPS,eff}(v)} = \sigma_{\text{GPS}} \cdot (1 + G \cdot e^{-|v|/v_s}) \quad (16)$$

where $G = \text{gps_low_speed_gain}$ (default 10.0) and $v_s = \text{gps_low_speed_v_scale}$ (default 2.0 m/s). The measurement covariance becomes:

$$\mathbf{R}_{\text{GPS}(v)} = \begin{pmatrix} \sigma_{\text{GPS,eff}}^2 & 0 \\ 0 & \sigma_{\text{GPS,eff}}^2 \end{pmatrix} \quad (17)$$

At standstill ($v \approx 0$): $\sigma_{\text{eff}} = 2.0 \cdot 11 = 22 \text{ m} \rightarrow K_{\text{GPS}} \approx 0.01$ (nearly ignores GPS). At highway speed ($v > 10 \text{ m/s}$): $\sigma_{\text{eff}} \approx \sigma_{\text{GPS}}$ (unchanged).

Speed	Effective σ_{GPS}	K_{GPS} (approx.)
0 m/s (standstill)	22.0 m	0.01
2 m/s (crawling)	10.1 m	0.05
5 m/s (parking)	3.6 m	0.28
10 m/s (urban)	2.1 m	0.54
>15 m/s	$\approx 2.0 \text{ m}$	0.56

Table 2: Speed-dependent GPS noise scaling. Gain $G = 10$, $v_s = 2 \text{ m/s}$, $\sigma_{\text{GPS}} = 2.0 \text{ m}$.

Physical rationale: At standstill, GPS receivers exhibit 1–3 m multipath jitter from building reflections. The CTRV model predicts “stay put” but each GPS update shifts the position by the jitter amount. Without dynamic scaling, the filter follows this jitter (producing a visible zig-zag). With $G = 10$, the effective measurement noise at $v = 0$ is 22 m, causing the filter to almost entirely ignore GPS updates — relying on the (correct) zero-speed prediction.

The transition is smooth: at typical parking-lot speeds (2–5 m/s) the filter already regains 30% GPS trust, which is appropriate for the lower multipath environment at non-zero velocity.

The 2×2 innovation covariance $\mathbf{S} = \mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R}(v)$ is inverted analytically (closed-form 2×2 inverse) for performance.

5.2 Wheel Speed (Dense, Full Rate)

$$z_v = v_{\text{wheel}}, \quad H_v = (0 \ 0 \ 1 \ 0 \ 0), \quad R_v = \sigma_v^2 \cdot s_v[k] \quad (18)$$

This is a scalar update: only the speed state is directly observed. The Kalman gain simplifies to $\mathbf{K}_v = \mathbf{P}_{:,2} / (P_{2,2} + R_v[k])$.

The per-sample scale factor $s_v[k]$ defaults to 1.0 under normal conditions and is inflated during ABS braking (see Section 11).

5.3 Yaw Rate (Dense, Full Rate)

$$z_\omega = \omega_{\text{gyro}}, \quad H_\omega = (0 \ 0 \ 0 \ 0 \ 1), \quad R_\omega = \sigma_\omega^2 \cdot s_\omega[k] \quad (19)$$

Similarly scalar: $\mathbf{K}_\omega = \mathbf{P}_{:,4} / (P_{4,4} + R_\omega[k])$.

Like s_v , the scale $s_\omega[k]$ is 1.0 normally and inflated during ABS events when the yaw rate source is unreliable (see Section 11).

6 Rauch–Tung–Striebel Smoother

For offline (batch) processing, the RTS smoother [1] runs a backward pass after the forward filter, incorporating future measurements to refine past estimates:

$$\mathbf{G}_k = \mathbf{P}_k^f \mathbf{F}_{k+1}^\top (\mathbf{P}_{k+1}^p)^{-1} \quad (20)$$

$$\hat{\mathbf{x}}_k^s = \hat{\mathbf{x}}_k^f + \mathbf{G}_k (\hat{\mathbf{x}}_{k+1}^s - \hat{\mathbf{x}}_{k+1}^p) \quad (21)$$

$$\mathbf{P}_k^s = \mathbf{P}_k^f + \mathbf{G}_k(\mathbf{P}_{k+1}^s - \mathbf{P}_{k+1}^p)\mathbf{G}_k^\top \quad (22)$$

where superscripts f , p , s denote filtered, predicted, and smoothed estimates respectively.

The smoother eliminates filter lag and produces the minimum-variance estimate given **all** data (past and future). For vehicle trajectories, this results in smooth paths through curves without the overshoot typical of causal filters.

7 Coordinate System

7.1 WGS84 ↔ Local ENU Conversion

GPS coordinates (longitude, latitude in degrees) are converted to a local tangent-plane approximation:

$$x_{\text{ENU}} = (\text{lon} - \text{lon}_0) \cdot M_{\text{lon}(\text{lat}_0)} \quad (23)$$

$$y_{\text{ENU}} = (\text{lat} - \text{lat}_0) \cdot M_{\text{lat}(\text{lat}_0)} \quad (24)$$

where $M_{\text{lat}(\varphi)}$ and $M_{\text{lon}(\varphi)}$ are the WGS84 ellipsoidal meters-per-degree of latitude and longitude at reference latitude $\varphi = \text{lat}_0$ (Snyder, **Map Projections: A Working Manual**, USGS Professional Paper 1395, 1987):

$$M_{\text{lat}(\varphi)} = 111\,132.92 - 559.82 \cos(2\varphi) + 1.175 \cos(4\varphi) - 0.0023 \cos(6\varphi) \quad (25)$$

$$M_{\text{lon}(\varphi)} = 111\,412.84 \cos(\varphi) - 93.5 \cos(3\varphi) + 0.118 \cos(5\varphi) \quad (26)$$

implemented as `meters_per_degree_lat()` / `meters_per_degree_lon()` in `constants.py`. Unlike the single global constant `EARTH_METERS_PER_DEGREE = 111,319.49 m/°` (kept for backward compatibility — it equals $M_{\text{lon}(0)}$, the equatorial value), these two functions are accurate across the full latitude range: true meters-per-degree of latitude varies from about 110,574 m at the equator to 111,694 m at the poles, a $\approx 1\%$ spread that a single constant cannot capture. Every coordinate-conversion site in `trajkit` (`kalman.py`, `jax_kalman.py`, `clothoid.py`'s `gps_enu()`, and the sampling/repair helpers in `processor.py`) evaluates $M_{\text{lat}}/M_{\text{lon}}$ at the locally relevant reference latitude — typically the track's first fix, or the segment mean where a per-segment reference is more appropriate (e.g. `_calibrate_dt`, `_repair_gps_freezes`) — rather than using the fixed constant. The inverse conversion uses the same two functions evaluated at the same reference latitude, to produce the final output in geographic coordinates.

This flat-Earth approximation introduces negligible error for trajectories spanning < 50 km from the origin; the latitude-dependent $M_{\text{lat}}/M_{\text{lon}}$ remove the residual $\approx 0.1\text{--}0.2\%$ systematic scale error that a single global constant would otherwise introduce at non-equatorial latitudes.

8 Implementation Architecture

8.1 Numba JIT Compilation

The computational core `_ekf_forward_rts()` is a single `@njit`-decorated function [2] containing the forward loop (predict + 3 updates per step) and the backward RTS loop. This design:

- Avoids Python interpreter overhead for the $O(n)$ tight loop
- Keeps all intermediate arrays (states, covariances, Jacobians) in Numba's native memory layout
- Achieves **14× speedup** over equivalent pure-Python code

Backend	$n = 853$ k	Speedup
Pure Python	58.0 s	1×
Numba (CPU)	4.1 s	14×
JAX (CPU)	7.7 s	7.5×

Table 3: Runtime comparison for the full EKF + RTS pipeline.

8.2 Class Structure

```
class GPSKalmanFilter:
    def __init__(self, config: KalmanConfig) -> None
    def run(self, longitude, latitude, v_reference_ms,
            yaw_rate_rad, gps_update_mask, heading_init=None
            ) -> KalmanResult
```

The `run()` method performs:

1. Coordinate conversion (WGS84 $\rightarrow$ ENU)
2. Initial heading estimation from first two GPS fixes
3. Forward EKF + optional RTS (via `_ekf_forward_rts`)
4. Inverse coordinate conversion (ENU $\rightarrow$ WGS84)

8.3 JAX Backend Parity

`jax_kalman.py` provides `JAXKalmanFilter` as an XLA/GPU-capable alternative (`smoothing='kalman_jax'`), sharing the same `KalmanConfig` and `KalmanResult` types. It is now numerically aligned with `GPSKalmanFilter` on every mechanism this document describes:

- **Position-drift fix:** the JAX process-noise matrix $\mathbf{Q}$ includes the $\sigma_{\text{drift}}^2 \cdot \Delta t$ term from Section 3.2.
- **Speed-dependent GPS noise:** the JAX GPS measurement covariance is recomputed every step from the post-update speed state using the same $\sigma_{\text{GPS,eff}(v)}$ formula as Table 2 (`gps_low_speed_gain`, `gps_low_speed_v_scale`).
- **Per-sample ABS noise scaling:** `JAXKalmanFilter.run()` accepts `speed_noise_scale` / `yaw_rate_noise_scale` / `process_noise_scale` with identical semantics to `GPSKalmanFilter.run()` — the per-sample arrays are threaded through `jax.lax.scan` as scan inputs (rather than closed-over scalars) so each step uses its own noise multiplier, mirroring `r_v_arr` / `r_omega_arr` / `q_scale_arr` in the Numba implementation (Section 11).

A direct numerical comparison on synthetic tracks — including a standstill segment with sparse GPS ($f_s/f_{\text{GPS}} = 100$) and an ABS-style speed-noise inflation window — showed agreement to within floating-point precision ($< 5 \times 10^{-13}$ m) between the two backends on all three scaling arguments, versus measurable systematic divergence before this change.

JAX-only extension: `JAXKalmanFilter.run()` additionally accepts `yaw_rate_mask`, a boolean array with no Numba equivalent. Where `False`, the yaw-rate measurement update is skipped entirely for that sample (a hard gate), rather than merely down-weighted as `yaw_rate_noise_scale` does. On a synthetic test with a 400-sample yaw sensor fault (a false +2 rad/s offset), masking the corrupted window kept the peak heading error at 0.002° versus 48° without masking. This is useful when fault detection can identify **specific** bad samples (e.g. sensor self-test flags) rather than a general confidence level.

Both backends can now be used interchangeably for every scenario this document covers, including ABS events.

9 Tuning Parameters

Parameter	Default	Effect of increasing
$\sigma_{\text{pos,GPS}}$	2.0 m	Less trust in GPS → smoother path
$\sigma_{\text{v,wheel}}$	0.1 m/s	Less trust in wheel speed → follows GPS-derived speed
σ_{ω}	0.2 rad/s	Less trust in gyro → heading follows GPS
σ_a	2.0 m/s ²	Allows faster speed changes → better acceleration tracking
$\sigma_{\dot{\omega}}$	0.5 rad/s ²	Allows faster yaw changes → better cornering
σ_{drift}	5.0 m/ $\sqrt{s}$	Stronger GPS pull → prevents dead-reckoning drift
gps_low_speed_gain	10.0	More GPS suppression at standstill → less multipath jitter
gps_low_speed_v_scale	2.0 m/s	Wider transition region → GPS suppressed at higher speeds
steering_ratio	15.5	Steering wheel → road wheel gear ratio (bicycle model)
wheelbase_m	2.68 m	Vehicle wheelbase for yaw rate derivation

Table 4: Tuning parameters and their effects on filter behavior.

9.1 Speed Bias from RTS Smoothing

The RTS smoother tends to “cut corners” in curves—the smoothed trajectory is shorter than the actual path, causing a systematic speed underestimate:

$$\Delta v \approx -v \cdot \kappa \cdot \delta_{\text{lateral}} \quad (27)$$

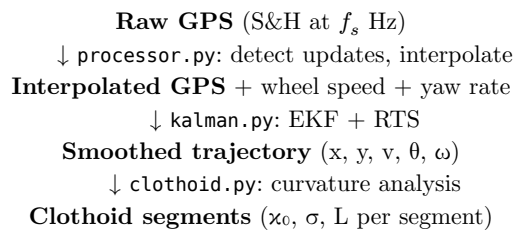
$\sigma_{\text{v,wheel}}$	Mean bias	Std	Bias at 200+ km/h
0.5 (old)	−4.7 km/h	5.4 km/h	−9.3 km/h
0.1 (new)	−0.2 km/h	1.6 km/h	−0.4 km/h

Table 5: Speed bias reduction by tightening wheel-speed noise.

Reducing $\sigma_{\text{v,wheel}}$ from 0.5 to 0.1 forces the filter to trust the wheel speed sensor more closely, suppressing the corner-cutting artifact.

10 Data Flow

The module fits into the larger GPS Track Conditioning pipeline:



10.1 Input Requirements

- longitude, latitude: Interpolated GPS (S&H removed, freeze-repaired)
- v_reference_ms: Wheel speed at full sample rate (f_s)

- **yaw_rate_rad**: Signed yaw rate at full sample rate. Derived from steering angle via bicycle model: $\omega = v \cdot \tan(\delta_{\text{steering}}/i_s)/L$ where $i_s = 15.5$ (steering ratio) and $L = 2.68$ m (wheel-base). Alternatively from ESP gyroscope when available (rare in practice)
- **gps_update_mask**: Boolean mask marking actual GPS measurement epochs

10.2 Output: `KalmanResult`

Field	Type	Description
<code>x_m, y_m</code>	<code>NDArray</code>	Position in local ENU frame [m]
<code>speed_ms</code>	<code>NDArray</code>	Filtered speed [m/s]
<code>heading_rad</code>	<code>NDArray</code>	Filtered heading [rad]
<code>yaw_rate_rad</code>	<code>NDArray</code>	Filtered yaw rate [rad/s]
<code>longitude, latitude</code>	<code>NDArray</code>	Converted back to WGS84 [°]
<code>P_diag</code>	<code>NDArray (n,5)</code>	State covariance diagonal

Table 6: Fields of the `KalmanResult` output structure.

11 ABS Braking Robustness

During ABS (Anti-lock Braking System) intervention, the wheel speed signal oscillates at ≈ 15 Hz due to repeated lock/unlock cycles. Without treatment, three mechanisms corrupt the EKF:

1. **Bicycle-model yaw rate is invalid**: $\omega = v \cdot \tan(\delta)/L$ uses oscillating v , producing error/ σ ratios of $\approx 3.5 \times$. With $\sigma_\omega = 0.2$ rad/s, the filter trusts the corrupted ω and heading drifts up to 8° over a 2 s ABS event.
2. **Wheel speed is unreliable**: The raw signal swings ± 10 km/h around the true deceleration curve.
3. **CTRV prediction is wrong**: With invalid ω and v , position error accumulates ≈ 0.28 m per prediction epoch between GPS corrections.

The implementation addresses these with two independent mechanisms: a speed-interpolation stage that runs unconditionally when `abs_flag` is provided, and a per-sample measurement-noise inflation that the EKF applies during the update step. An earlier design additionally crossfaded the **position** output toward a model-free Butterworth track during ABS events; that position-blend stage was dropped in favour of the noise-inflation approach below and is no longer part of the pipeline.

11.1 Stage 1: Speed Interpolation

The method `_clamp_speed_during_abs()` replaces wheel speed during ABS epochs with a linear interpolation between the last non-ABS sample before and the first non-ABS sample after each event:

$$v_{\text{smooth}}[k] = \begin{cases} v_{\text{raw}}[k] & \text{if ABS inactive} \\ \text{interp}(k, \{j : \text{ABS}[j] = 0\}, v_{\text{raw}}) & \text{if ABS active} \end{cases} \quad (28)$$

This removes the 15 Hz oscillation while preserving the deceleration trend (offline processing knows both anchor points). The interpolated v_{smooth} is used for both the bicycle-model yaw rate computation and as the Kalman speed measurement input.

11.2 Stage 2: Wheel-Speed Measurement-Noise Inflation

Rather than crossfading the **position** toward a model-free filter, the implementation keeps the full GPS + wheel-speed + yaw-rate fusion active throughout the ABS event and instead tells the

EKF to distrust the interpolated wheel speed while it lasts. `_build_abs_noise_scale()` produces a per-sample multiplier $s_v[k] \in [1, s_{\max}]$ that scales the wheel-speed measurement variance $R_v[k] = \sigma_{v,\text{wheel}}^2 \cdot s_v[k]$ (see Table 4 and the R_v definition in Section 5). With the measurement variance inflated, the Kalman gain for the speed update drops toward zero and the EKF lets GPS position updates and the CTRV model carry the speed estimate through the ABS window instead.

The yaw rate is left untouched: when it comes from a direct sensor it remains valid under tire saturation, and the bicycle-model-derived yaw rate already benefits from v_{smooth} in Stage 1.

Temporal margin: because the ABS flag typically lags the actual wheel oscillation, the inflated-noise window is dilated by a fixed margin before and after each contiguous ABS run:

- **Pre-ABS margin:** `_ABS_PRE_MARGIN_S` (default 0.3 s)
- **Post-ABS margin:** `_ABS_POST_MARGIN_S` (default 0.5 s)

Raised-cosine ramp: each window edge is smoothed over `_ABS_RAMP_S` (default 0.15 s) with a half-wave cosine,

$$s_v(t) = 1 + (s_{\max} - 1) \cdot \frac{1}{2} (1 - \cos(\pi \cdot t / t_{\text{ramp}})), \quad (29)$$

avoiding a step change in measurement trust that would otherwise inject a kink into the estimate.

Peak inflation: `_ABS_SPEED_NOISE_INFLATION` defaults to 400 (variance scale), i.e. a $20 \times$ inflation of $\sigma_{v,\text{wheel}}$.

Advantages of noise inflation over a position blend:

- No separate model-free filter branch, no blend mask, no state-injection logic at re-entry — the EKF’s own covariance propagation handles the transition.
- The full sensor fusion (GPS + yaw rate) stays active during ABS instead of temporarily disabling it in favour of a position-only filter.
- Simpler to reason about: a single scalar multiplier on one measurement channel, versus a parallel filter, a crossfade schedule, and a re-entry correction.

Current scope: the ABS handling itself only ever inflates the wheel-speed channel automatically, by design — not as an unfinished feature. A direct-sensor yaw rate stays valid under tire saturation, and a bicycle-model-derived one already benefits from the Stage-1 speed interpolation, so there is no ABS-specific reason to distrust it; inflating the full process-noise Q would reduce trust in **every** measurement rather than the one channel actually at fault. Both `GPSKalmanFilter.run()` and `JAXKalmanFilter.run()` nonetheless accept per-sample `yaw_rate_noise_scale` and `process_noise_scale` arrays (with identical semantics on both backends — Section 8.3), and `GPSProcessor.process()` exposes them as optional pass-through parameters for callers with an independent reason to distrust specific samples — e.g. a yaw sensor self-test flag, or a manoeuvre poorly captured by the CTRV model — unrelated to ABS.

Parameter	Default
<code>_ABS_FLAG_THRESHOLD</code>	0.5
<code>_ABS_SPEED_NOISE_INFLATION</code>	400.0 (variance, $\approx 20 \times$ in σ)
<code>_ABS_PRE_MARGIN_S</code>	0.3 s
<code>_ABS_POST_MARGIN_S</code>	0.5 s

Parameter	Default
<code>_ABS_RAMP_S</code>	0.15 s

Table 7: ABS noise-inflation constants (`constants.py`).

No end-to-end accuracy benchmark for this mechanism is available yet; Table 7 documents the current parameters rather than measured performance. A benchmark analogous to the earlier position-blend study (full stops with a steering-angle-derived yaw rate) would be a useful follow-up.

12 GPS Freeze Repair (Preprocessing)

Before coordinates reach the EKF, `processor.py` applies `_repair_gps_freezes()` to handle a specific GPS receiver pathology: **single-axis coordinate freezes**.

12.1 Phenomenon

Some GPS receivers intermittently freeze one coordinate axis—for example, reporting the same longitude 10.78352077° for 5–8 consecutive epochs while latitude updates normally. At 120 km/h, the vehicle travels ≈ 2.5 m per GPS fix in the frozen direction. The receiver reports no movement in that axis, creating a false purely-northward trajectory segment.

Consequences without repair:

- The EKF partially tracks the frozen longitude, introducing a heading kink when the freeze ends
- The heading distortion corrupts downstream curvature derivation ($\kappa = d\theta/ds$) and clothoid segmentation
- The RTS smoother propagates the kink globally (it is not local)

12.2 Detection: Innovation-Based Criterion

A coordinate is classified as **frozen** when ALL of the following hold:

1. The axis value is unchanged for $\geq \text{min_run_length}$ consecutive GPS epochs (default: 3)
2. The vehicle speed exceeds `min_speed` (default: 5.0 m/s)
3. The **expected** displacement in the frozen axis—computed from heading and arc distance—exceeds `min_expected_change_m` (default: 5.0 m)

The third criterion is critical: in tight curves, one coordinate legitimately stays constant for several epochs when the vehicle moves perpendicular to that axis (e.g., driving North $\rightarrow$ longitude barely changes). The heading-projection test eliminates these **false positives**.

Condition	Tight curve (valid)	True freeze (invalid)
Axis unchanged	✓ (3+ epochs)	✓ (5+ epochs)
Speed > 5 m/s	✓	✓
Heading predicts movement	✗ (heading $\perp$ axis)	✓ (heading $\parallel$ axis)
Classified as freeze?	No	Yes

Table 8: Innovation check distinguishes true freezes from normal quantization.

12.3 Repair Strategy: Linear Interpolation

For each confirmed freeze run, the frozen coordinate values are replaced with a linear interpolation between the last valid epoch before the freeze and the first valid epoch after:

$$\text{lon}_{\text{repaired}}(t) = \text{lon}_{\text{before}} + (\text{lon}_{\text{after}} - \text{lon}_{\text{before}}) \cdot \frac{t - t_{\text{before}}}{t_{\text{after}} - t_{\text{before}}} \quad (30)$$

The **valid** axis is preserved unchanged. The EKF then receives the repaired coordinates with standard measurement noise σ_{GPS}^2 for both axes—maintaining full Kalman gain and position anchoring.

12.4 Why Not Partial (1D) Measurement Updates?

An alternative approach was tested: inflate the GPS noise for the frozen axis to $\approx \infty$ (or $100 \times \sigma^2$), giving the Kalman zero (or reduced) gain for that axis while preserving the valid axis at full gain. This is the textbook “partial observability” solution.

Result: it performs significantly worse (tight curves: +3.4 m vs +0.5 m median error) because:

1. Reduced gain $\rightarrow$ more position drift during the freeze forward pass
2. The RTS backward smoother is **global**: drift in one region propagates to the entire trajectory
3. Tight curves far from the freeze region are affected by the rebalancing

The interpolation approach works better because it **maintains position anchoring** (the linear bridge limits drift) while providing a smooth signal that the RTS smoother treats as unremarkable.

Approach	Tight Δ	Moderate Δ	Straight Δ
No fix (baseline)	0.00 m	0.00 m	0.00 m
Interpolation (current)	+0.53 m	−0.67 m	+0.02 m
Epoch removal	+0.63 m	−0.56 m	+0.01 m
Noise inflation (100×)	+3.42 m	−0.69 m	+0.03 m
Noise inflation (∞)	+4.15 m	−0.67 m	+0.02 m

Table 9: Median error Δ vs baseline for GPS freeze repair approaches (G2 clothoid chain, `kappa_clip=None`).

12.5 Code Reference

```
@staticmethod
def _repair_gps_freezes(
    lon, lat, update_idx, speed,
    min_speed=5.0, min_run_length=3,
    min_expected_change_m=5.0,
) -> tuple[NDArray, NDArray]:
    """Detect and interpolate single-axis GPS freezes."""
```

Called inside `GPSProcessor._apply_kalman()` after S&H interpolation and update detection, before building the GPS mask and calling `GPSKalmanFilter.run()`.

Note on the in-code docstring: the current docstring of `_repair_gps_freezes()` describes a noise-inflation / partial 1D measurement update — the approach this section just showed to perform **worse** (see Table 9). That description does not match the function body, which performs the linear-interpolation repair documented above. Treat this document as authoritative; the docstring should be corrected to match the implementation.

13 Summary

The `kalman.py` module provides a production-ready EKF/RTS implementation tailored to the specific challenges of high-rate vehicle GPS processing:

1. **Multi-rate fusion:** Dense wheel speed and gyro ($f_s \approx 500$ Hz) with sparse GPS (5–20 Hz) in a unified framework
2. **Position drift fix:** Custom process noise scaling for high sample rates (see Section 3.2)
3. **ABS braking robustness:** Source-dependent per-sample noise inflation with temporal margin and cosine ramp (see Section 11)
4. **GPS freeze repair:** Innovation-based detection and linear interpolation of single-axis coordinate freezes (see Section 12)
5. **Offline optimality:** RTS smoother gives minimum-variance estimate from all data
6. **Numba acceleration:** 4.1 s for 853k samples (see Table 3)
7. **Clean API:** Single `run()` call with geographic I/O, internal ENU math

References

- [1] H. E. Rauch, F. Tung, and C. T. Striebel, “Maximum likelihood estimates of linear dynamic systems,” *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965, doi: 10.2514/3.3166.
- [2] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, ACM, 2015, pp. 1–6. doi: 10.1145/2833157.2833162.